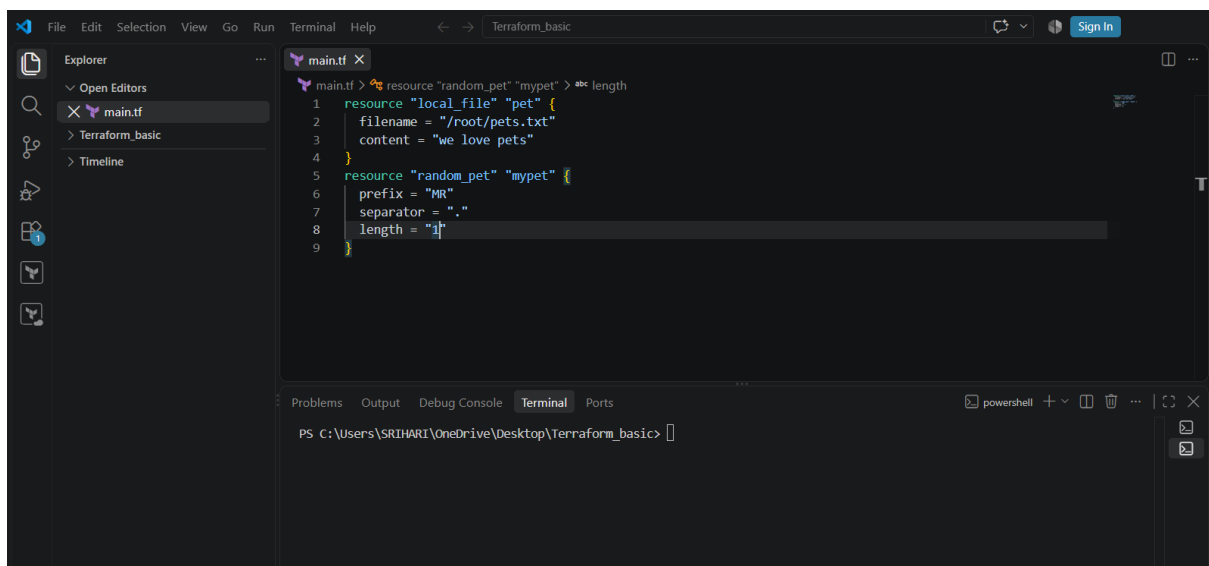


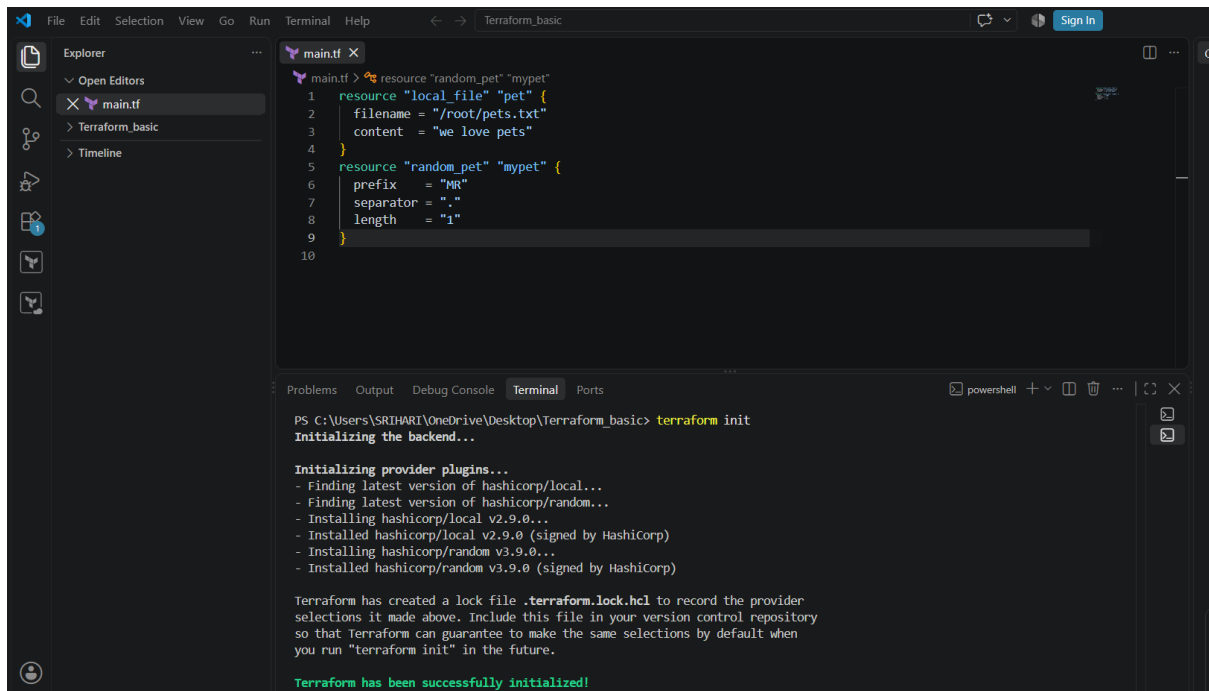
TERRAFORM – 02 – TASKS

1. Execute all the templates shown in the video.

- created main.tf.
- Defined two resources:
 - local_file.pet → creates /root/pets.txt
 - random_pet.mypet → generates a random name
- No Terraform initialization yet.



- terraform init is executed.
- Providers are downloaded:
 - hashicorp/local
 - hashicorp/random
- .terraform/ directory and .terraform.lock.hcl file are created.
- Terraform project is initialized successfully



```
main.tf
1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content = "we love pets"
4 }
5 resource "random_pet" "mypet" {
6   prefix = "MR"
7   separator = "."
8   length = "1"
9 }
10
```

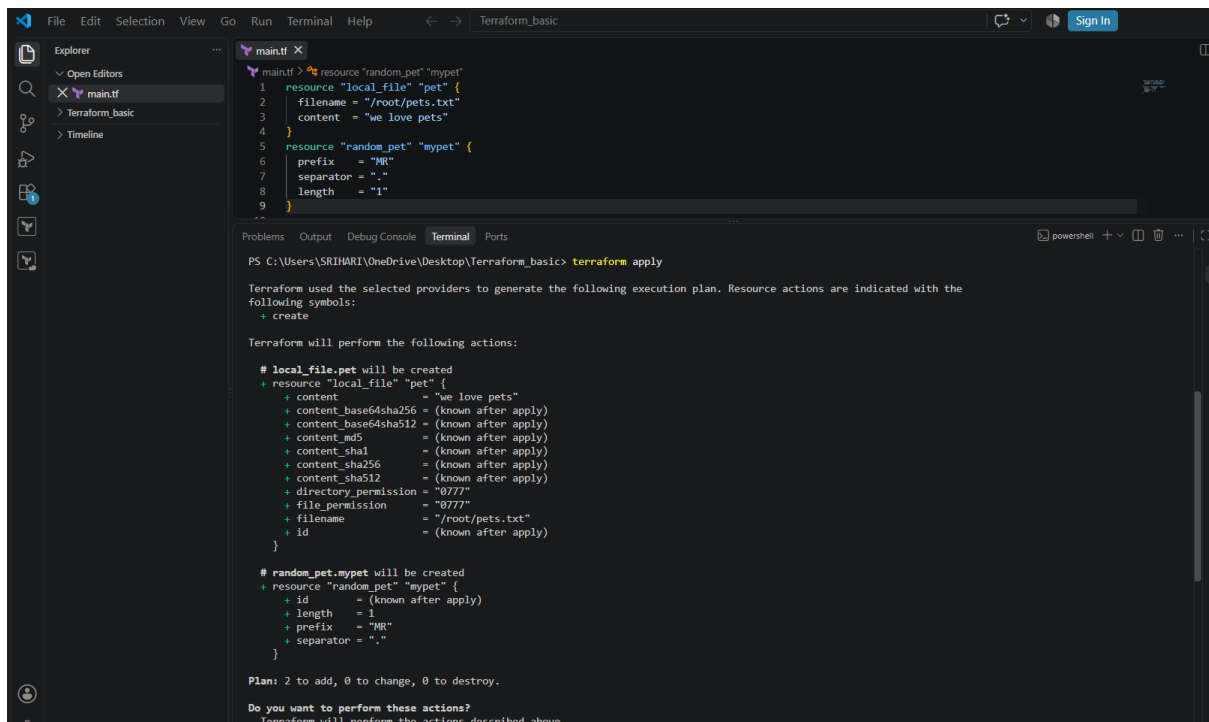
```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

- Terraform apply
- Terraform shows an execution plan.
- Both resources will be **created**.
- You type yes.
- pets.txt file is created.
- random_pet value is generated



```
main.tf
1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content = "we love pets"
4 }
5 resource "random_pet" "mypet" {
6   prefix = "MR"
7   separator = "."
8   length = "1"
9 }
```

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

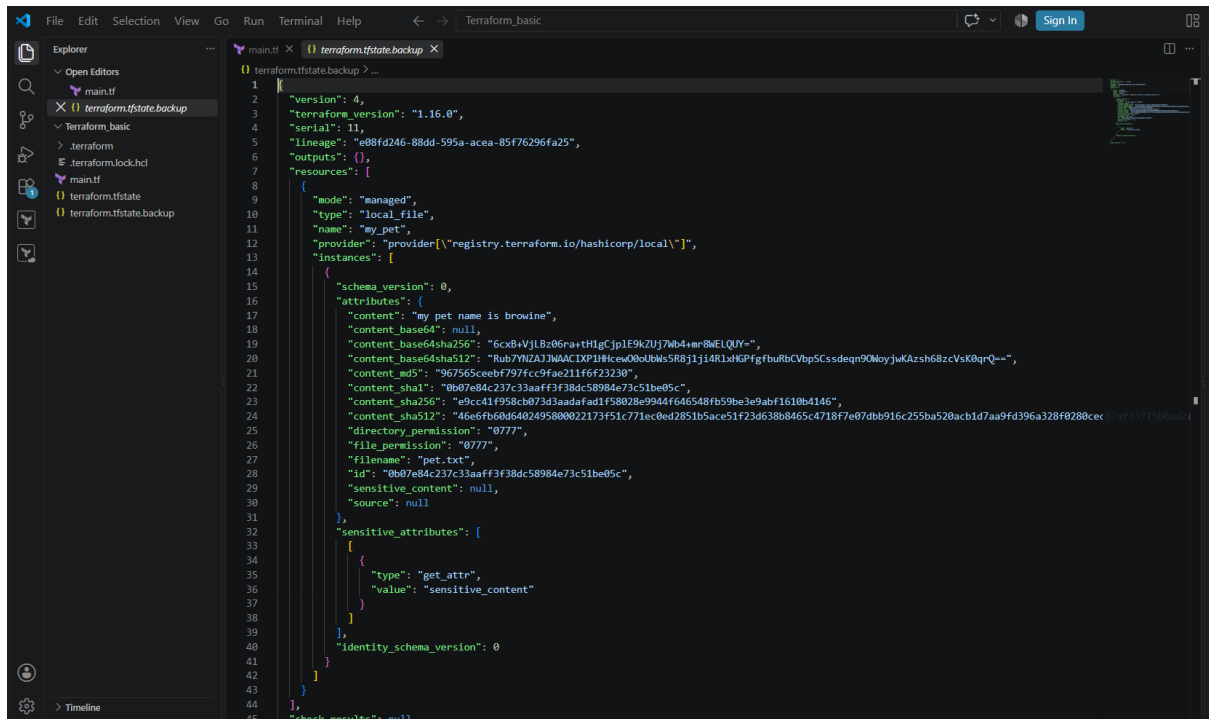
# local_file.pet will be created
+ resource "local_file" "pet" {
+ content = "we love pets"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "/root/pets.txt"
+ id = (known after apply)
}

# random_pet.mypet will be created
+ resource "random_pet" "mypet" {
+ id = (known after apply)
+ length = 1
+ prefix = "MR"
+ separator = "."
}

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
```

- terraform.tfstate file appears.
- State stores:
 - Resource IDs
 - File content hash
 - Provider metadata
- Terraform now **tracks real infrastructure**

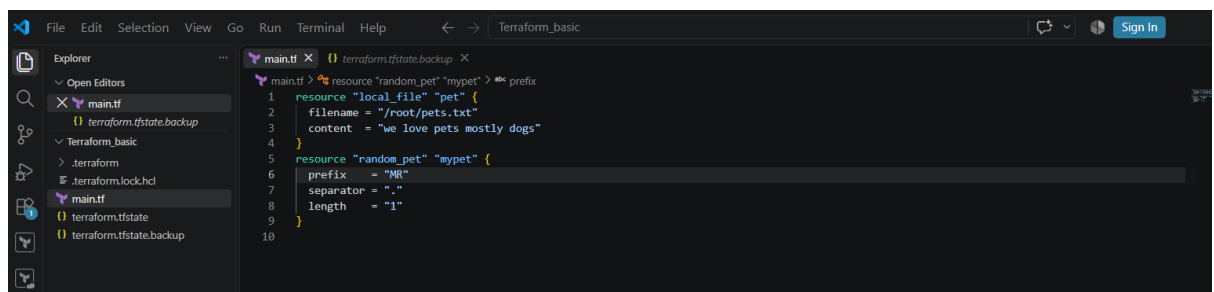


```

1 {
2   "version": 4,
3   "terraform_version": "1.16.0",
4   "serial": 11,
5   "lineage": "e08fd246-88dd-595a-acea-85f76296fa25",
6   "outputs": {},
7   "resources": [
8     {
9       "mode": "managed",
10      "type": "local_file",
11      "name": "my_pet",
12      "provider": "provider[\"registry.terraform.io/hashicorp/local\"],",
13      "instances": [
14        {
15          "schema_version": 0,
16          "attributes": {
17            "content": "my pet name is browine",
18            "content_base64": null,
19            "content_base64sha256": "6cx8+VjLz06ra+HlgCjplE9kZUj7Mb4+m8wELQUY=",
20            "content_base64sha512": "Rub7YNZAjJMAACIXP1lHcew00uUw5SR8j1ji4R1xHGpFgFbuRbCVbpSCsdeqn9OmoyjwKAZsh68zcVcK0qrQ==",
21            "content_md5": "967565ceebf797fcc9fae211f6f23230",
22            "content_sha1": "0b07e84c237c33aaff3f38dc58984e73c51be05c",
23            "content_sha256": "e9cc41f958cb073d3aadafad1f58028e9944f646548fb59be3e9abf1610b4146",
24            "content_sha512": "46e6fb60d6402495800022173f51c771ec0ed2851b5ace51f23d638b8465c4718f7e07dbb916c255ba520acb1d7a9af9d396a328f0280cec",
25            "directory_permission": "0777",
26            "file_permission": "0777",
27            "filename": "pet.txt",
28            "id": "0b07e84c237c33aaff3f38dc58984e73c51be05c",
29            "sensitive_content": null,
30            "source": null
31          },
32          "sensitive_attributes": [
33            {
34              "type": "get_attr",
35              "value": "sensitive_content"
36            }
37          ],
38          "identity_schema_version": 0
39        }
40      ],
41      "check_results": null
42    }
43  ],
44  "check_results": null
45 }

```

- modified content inside local_file.
- This causes a **ForceNew** change.
- Terraform detects drift between config and state



```

1 resource "random_pet" "mypet" {
2   prefix = "MR"
3   separator = "-"
4   length = 1
5 }
6
7 resource "local_file" "pet" {
8   filename = "/root/pets.txt"
9   content = "we love pets mostly dogs"
10 }

```

- Plan shows:
 - local_file.pet must be replaced
- Old file is destroyed.
- New file is created.

Key point

- Default behavior = **destroy → create**

```

PS C:\Users\SRIDHAR\OneDrive\Desktop\Terraform_basics> terraform apply
local_file.pet: Refreshing state... [id=d87a5018d4c1e1644d11eafcd53a95c8968ef]
random_pet.mypet: Refreshing state... [id=WR.trout]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/. destroy and then create replacement

Terraform will perform the following actions:

# local_file.pet must be replaced
./ resource "local_file" "pet" {
  ~ content      = "we love pets" -> "we love pets mostly dogs" # forces replacement
  ~ content_base64sha256 = "3d8pK8u7LP/4K513V16PtoKzrd4iul1k8etpxj0u" -> (known after apply)
  ~ content_base64sha512 = "t13QwccBj1K1zpAl8KcoaxC629M7d4y8houfDU23vcFHE45pdj1t02EwocK8g/80//1rVigD)w40Bk0liu--" -> (known after apply)
  ~ content_md5      = "3b16d6161272ff432c52407Aa07d4it" -> (known after apply)
  ~ content_sha1      = "d87a5018d4c1e1644d11eafcd53a95c8968ef" -> (known after apply)
  ~ content_sha256     = "2783692bcb82cfff0502e7d756597a3eda17ceb77896dc89019ada718f35" -> (known after apply)
  ~ content_sha512     = "b4827780c5e1418e2913ce902208a7780b6d6a11hof56e0db32f212e143536de77951c4e12a547638ad399138c71911ab604eff8ab565803270b0e0643080b" -> (known after apply)
  ~ id               = "d87a5018d4c1e1644d11eafcd53a95c8968ef" -> (known after apply)
  ~ (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.pet: Destroying... [id=d87a5018d4c1e1644d11eafcd53a95c8968ef]
local_file.pet: Destruction complete after 0s
local_file.pet: Creating...
local_file.pet: Creation complete after 0s [id=62d51ae81e9340f65350e961b509ecbc39430616]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

```

- added:
- lifecycle {
- create_before_destroy = true
- }

Key point

- Ensures **zero downtime replacement**.

```

1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content  = "we love pets mostly horses"
4   lifecycle {
5     create_before_destroy = true
6   }
7 }
8 resource "random_pet" "mypet" {
9   prefix      = "WR"
10  separator   = "."
11  length      = "1"
12 }
13

```

- New file is created first.

- Old file is destroyed after.

Key point

- Safe replacement strategy.

```

1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content  = "we love pets mostly horses"
4   lifecycle {
5     create_before_destroy = true
6   }
7 }
8 resource "random_pet" "mypet" {
9   prefix      = "MR"
10  separator   = "-"
11  length      = "1"
12 }
13

```

```

PS C:\Users\SRINANI\OneDrive\Desktop\Terraform_basics> terraform apply
local_file.pet: Refreshing state... [id=63d51ae0e0394065359e9611569ecch39430616]
random_pet.mypet: Refreshing state... [id=MR.trout]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+/- create replacement and then destroy

Terraform will perform the following actions:

# local_file.pet must be replaced
+/- resource "local_file" "pet" {
  ~ content      = "we love pets mostly dogs" -> "we love pets mostly horses" # forces replacement
  ~ content_base64sha256 = "s49z64JE91173pvcVmcQZCQ707807Gr66m4wH0XU" -> (known after apply)
  ~ content_base64sha512 = "8v3lfXQj4qWqQw4Pm4y2NEAx511A0Pmc78r4J5qYtTALeasJw412bR75AQH80M7H9Fe8J13JCj4msAg==" -> (known after apply)
  ~ content_md5      = "d89944cd3d6acc01f201a695f442c" -> (known after apply)
  ~ content_sha1     = "63d51ae0e0394065359e9611569ecch39430616" -> (known after apply)
  ~ content_sha256   = "b3ef6f65d40913d7577a69c53663994027ceec813eabd3a9dc08ac7d085" -> (known after apply)
  ~ content_sha512   = "4940f508528c3d6d60f9f9eae1c43265d64c4aac75a35c0074832673047e559ae061320820e69c71e35d9e01ed259033c87459f37d7def23237242a50eccc2" -> (known after apply)
  ~ id              = "63d51ae0e0394065359e9611569ecch39430616" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.pet: Creating...
local_file.pet: Creation complete after 0s [id=d1b5208b5e7b15607146f11b7f6ae0bf5278f4c5]
local_file.pet (deposed object 61042aba): Destroying... [id=63d51ae0e0394065359e9611569ecch39430616]
local_file.pet: Destruction complete after 0s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\SRINANI\OneDrive\Desktop\Terraform_basics>

```

- added:
- lifecycle {
- prevent_destroy = true
- }

Key point

- Protects critical resources from deletion.

```

1 resource "local_file" "pet" {
2   filename = "/root/pets.txt"
3   content  = "we love pets mostly horses & dogs"
4   lifecycle {
5     prevent_destroy = true
6   }
7 }
8 resource "random_pet" "mypet" {
9   prefix      = "MR"
10  separator   = "-"
11  length      = "1"
12 }
13

```

- Terraform refuses to destroy the resource.
- Apply fails if replacement is required.

Key point

- Manual intervention required to remove protection

```

1 resource "random_pet" "mypet" {
2   prefix = "my"
3   separator = "-"
4   length = 10
5 }
6
7 resource "local_file" "pet" {
8   filename = "/root/pets.txt"
9   content = "we love pets mostly horses & dogs"
10  lifecycle {
11    prevent_destroy = true
12  }
13 }

```

Problems Output Debug Console Terminal Ports

local_file.pet: Destruction complete after 0s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

PS C:\Users\SRDWA\OneDrive\Desktop\Terraform_basics> terraform apply

local_file.pet: Refreshing state... [id=dt9bb5e7b456071d6f1b7f6a0d8f5278f4c5]

random_pet.mypet: Refreshing state... [id=9L.trout]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- create

Terraform will perform the following actions:

```

# local_file.pet will be created
+ resource "local_file" "pet" {
+   content          = "we love pets mostly horses & dogs"
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5      = (known after apply)
+   content_sha1     = (known after apply)
+   content_sha256   = (known after apply)
+   content_sha512   = (known after apply)
+   directory_permission = "0777"
+   file_permission  = "0777"
+   filename         = "/root/pets.txt"
+   id               = (known after apply)
}

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.pet: Creating...

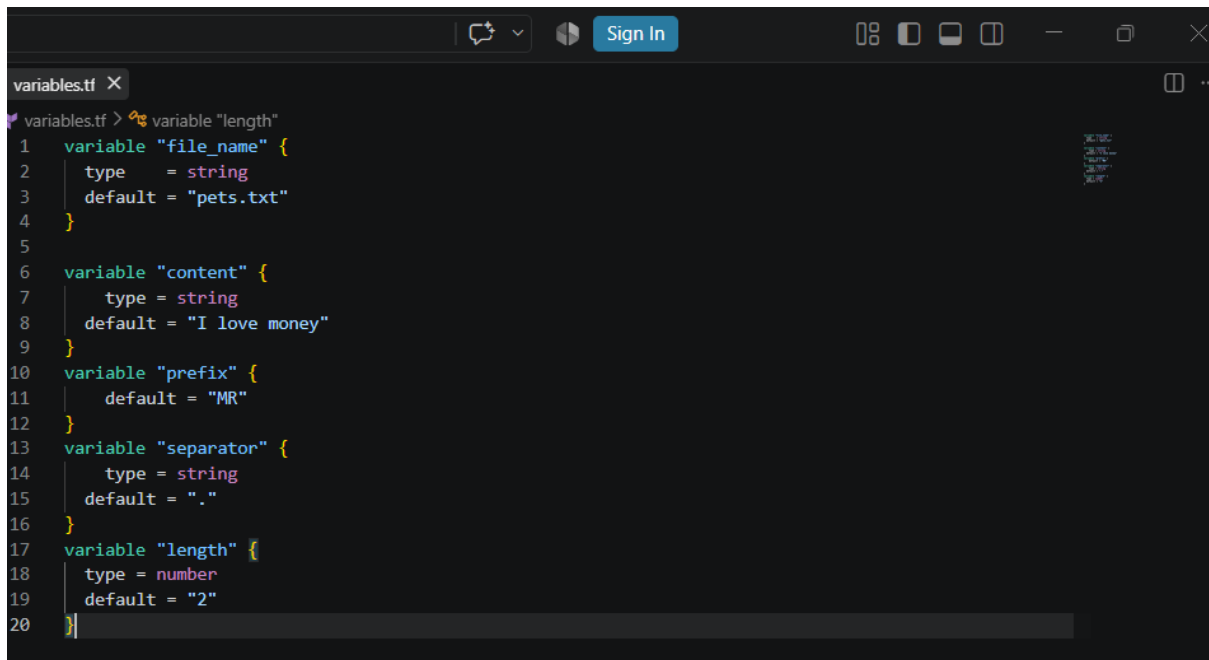
local_file.pet: Creation complete after 0s [id=b5370342d8c96da7eab84dc79249f4ec222a05a6]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

- Created a new file called **variables.tf**
- Defined reusable input variables:
 - file_name
 - content
 - prefix
 - separator
 - length

Key points

- Each variable has:
 - type
 - default
- Defaults allow Terraform to run **without passing values manually**

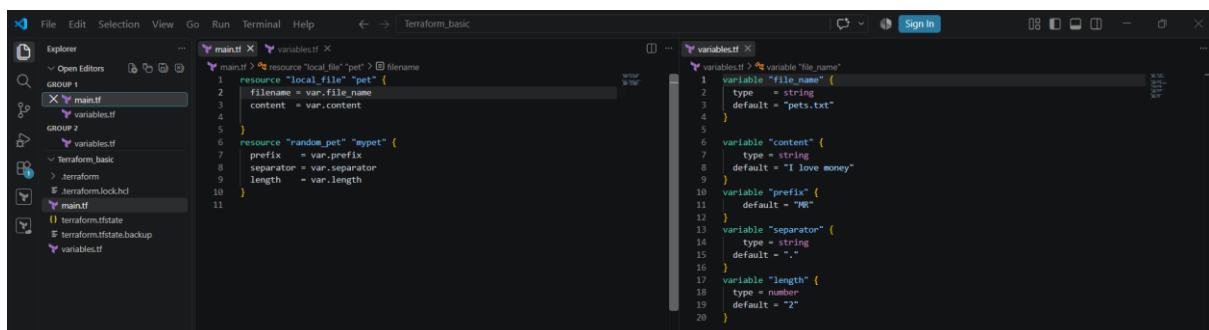


```
variables.tf
1  variable "file_name" {
2    type    = string
3    default = "pets.txt"
4  }
5
6  variable "content" {
7    type    = string
8    default = "I love money"
9  }
10 variable "prefix" {
11   default = "MR"
12 }
13 variable "separator" {
14   type    = string
15   default = "."
16 }
17 variable "length" {
18   type    = number
19   default = "2"
20 }
```

- Replaced hardcoded values with variables:
- filename = var.file_name
- content = var.content
- Updated random_pet resource:
- prefix = var.prefix
- separator = var.separator
- length = var.length

Key points

- var.<variable_name> is how Terraform references variables
- Configuration becomes **dynamic and reusable**



```
main.tf
1 resource "local_file" "pet" {
2   filename = var.file_name
3   content  = var.content
4 }
5
6 resource "random_pet" "mypet" {
7   prefix   = var.prefix
8   separator = var.separator
9   length  = var.length
10 }
11
```

```
terraform apply
```

Terraform shows:

```
# random_pet.mypet must be replaced
```

```
-/+ resource "random_pet" "mypet" {  
  ~ id      = "MR.trout"-> (known after apply)  
  ~ length  = 1-> 2 # forces replacement  
  # (2 unchanged attributes hidden)  
}
```

Plan: 2 to add, 0 to change, 2 to destroy.

Enter a value: yes

```
random_pet.mypet: Destroying... [id=MR.trout]
```

local_file.pet: Destroying... [id=b5370342d0c96da7ea8a4de79249faec222a05a6]

```
random_pet.mypet: Destruction complete after 0s
```

local_file.pet: Destruction complete after 0s

```
random_pet.mypet: Creating...
```

```
random_pet.mypet: Creation complete after 0s [id=MR.smiling.joey]
```

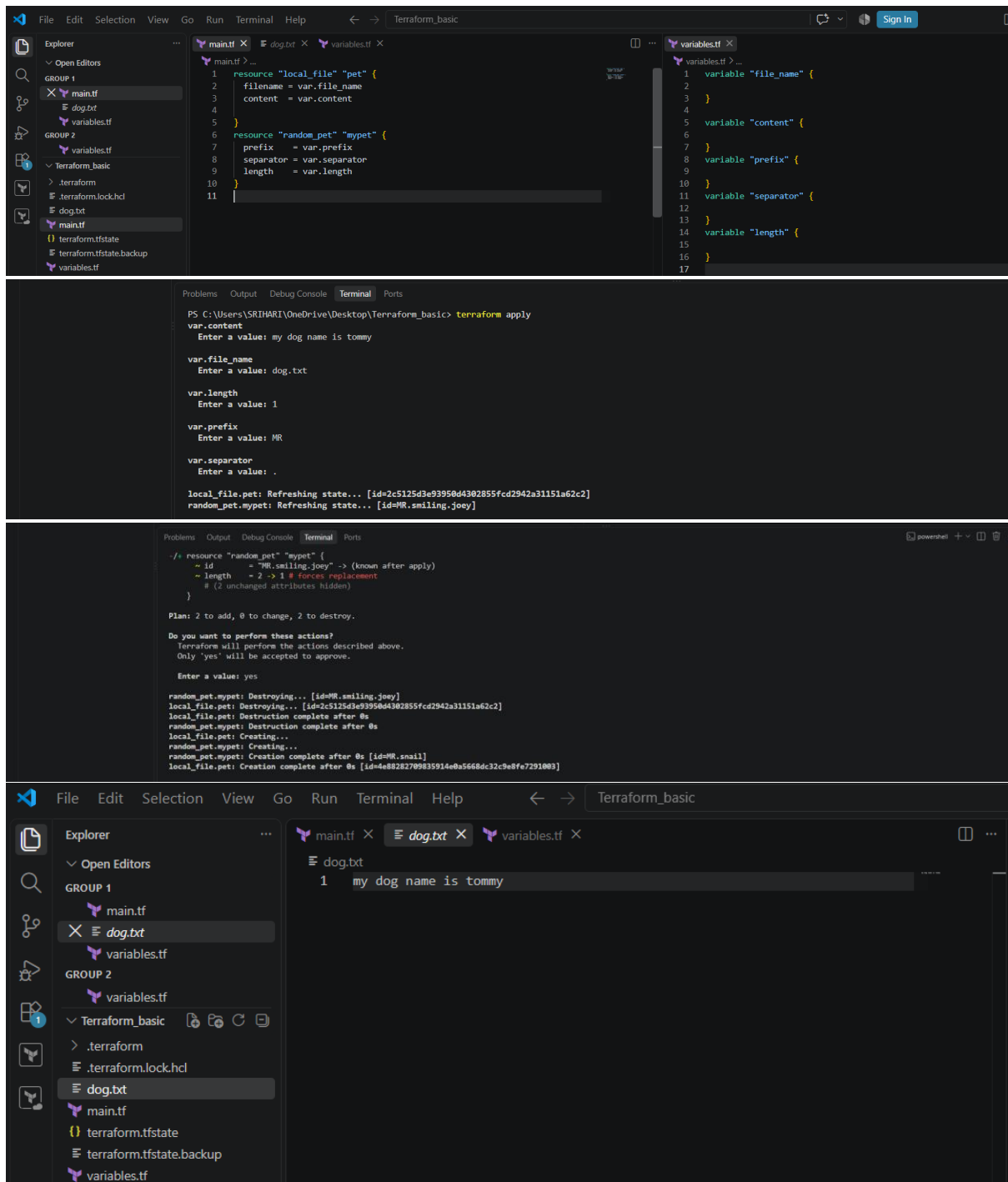
local_file.pet: Creating...

```
local_file.pet: Creation complete after 0s [id=2c5125d3e93950d4302855fcd2942a31151a62c2]
```

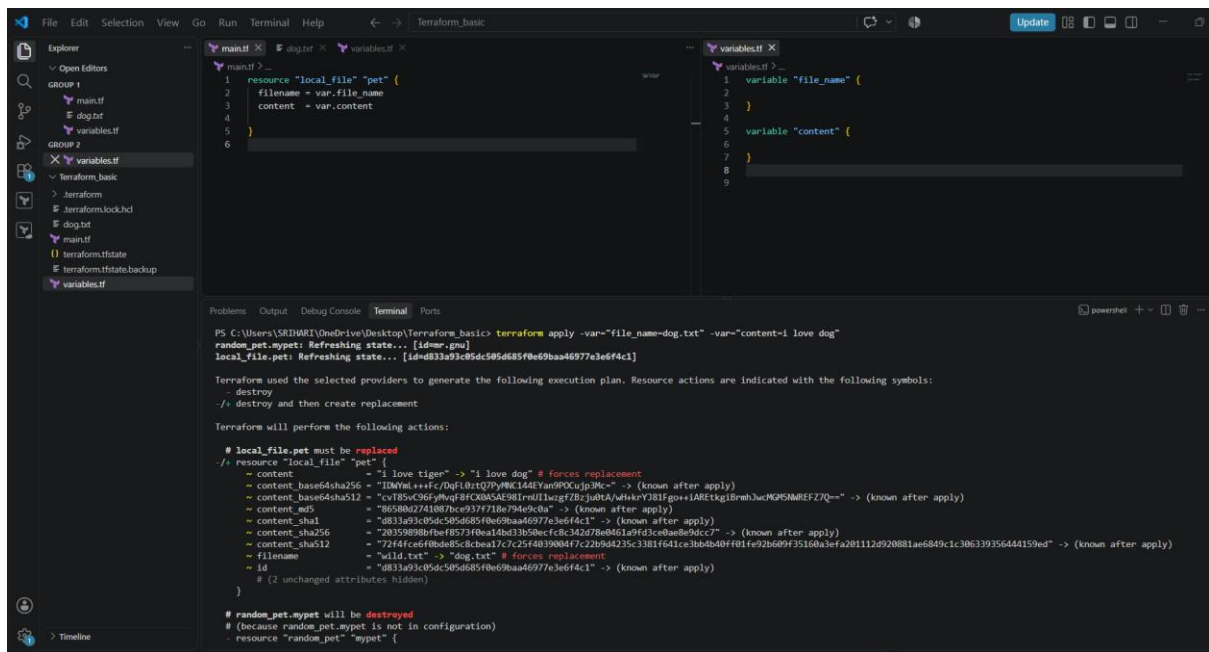
Content taken from variable

[illegible]

Next:



Next:

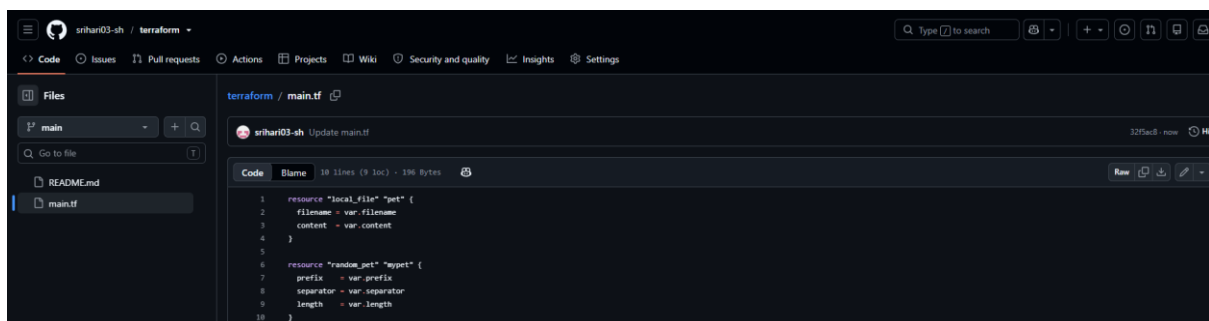


2.Integrate Terraform in Jenkins using the Terraform plugin

- opened main.tf directly in GitHub.
- local_file and random_pet resources reference variables using var.*.

Why this matters

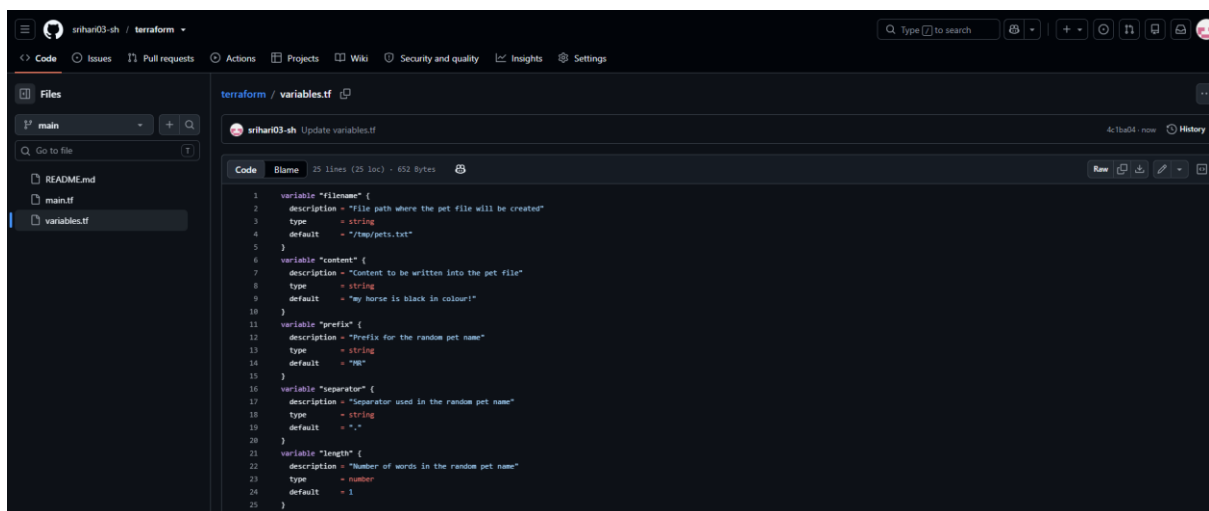
- No hard-coded values.
- Configuration is reusable and CI/CD friendly.



- defined all input variables:
 - filename (now /tmp/pets.txt)
 - content
 - prefix
 - separator
 - length

Why this matters

- Fixes Jenkins permission issue.
- Ensures Terraform config is valid



The screenshot shows a GitHub repository interface. On the left, the 'Files' sidebar shows a directory structure with 'main' selected. The main area displays the content of 'terraform / variables.tf'. The file is titled 'srihari03-sh Update variables.tf' and shows a commit history of 25 lines (25 loc) and 652 bytes. The code defines five variables: 'filename' (string, default '/tmp/pets.txt'), 'content' (string, default 'My horse is black in colour!'), 'prefix' (string, default 'PG'), 'separator' (string, default '.'), and 'length' (number, default 1).

```

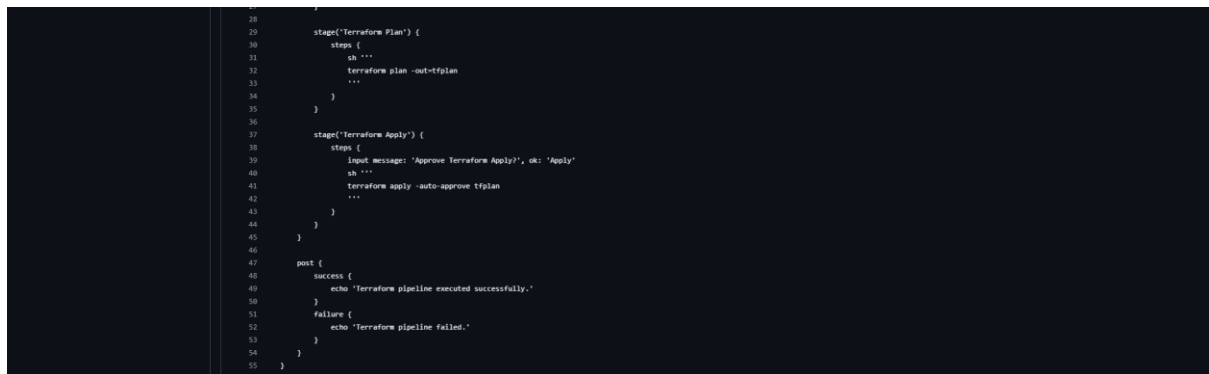
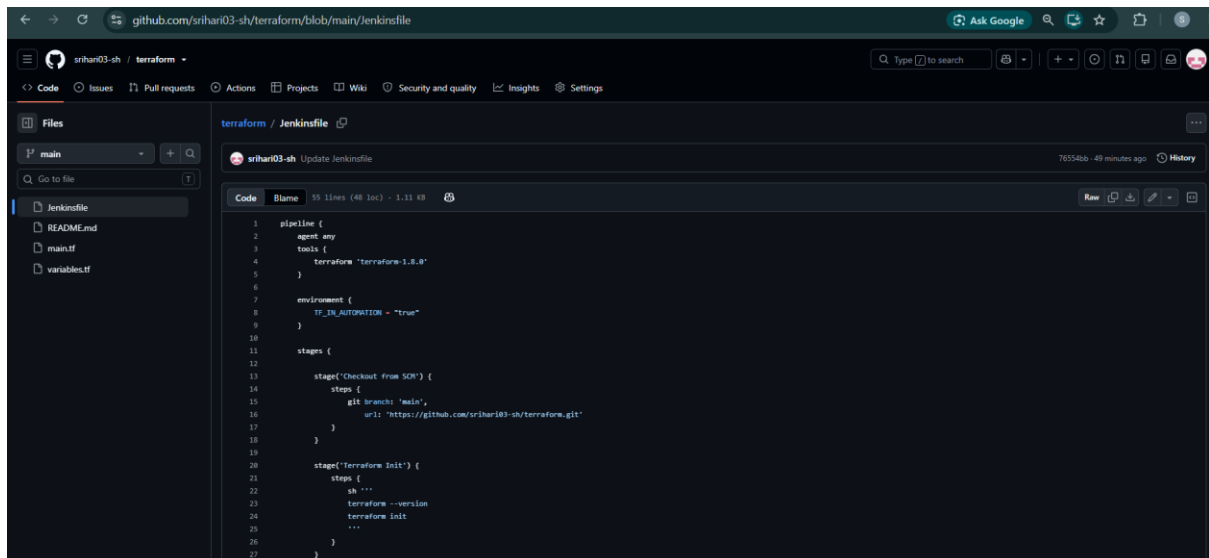
1 variable "filename" {
2   description = "File path where the pet file will be created"
3   type        = string
4   default     = "/tmp/pets.txt"
5 }
6 variable "content" {
7   description = "Content to be written into the pet file"
8   type        = string
9   default     = "My horse is black in colour!"
10 }
11 variable "prefix" {
12   description = "Prefix for the random pet name"
13   type        = string
14   default     = "PG"
15 }
16 variable "separator" {
17   description = "Separator used in the random pet name"
18   type        = string
19   default     = "."
20 }
21 variable "length" {
22   description = "Number of words in the random pet name"
23   type        = number
24   default     = 1
25 }

```

- created a file named **Jenkinsfile**
- Pipeline stages included:
 - Checkout
 - Terraform Init
 - Terraform Plan
 - Terraform Apply (manual approval)

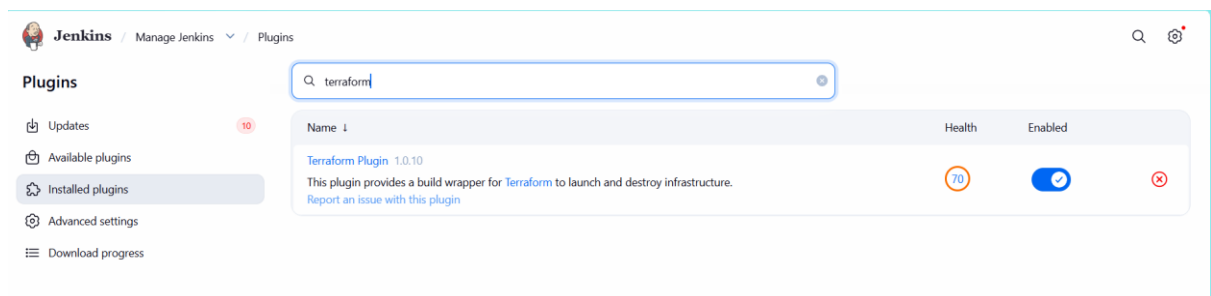
Why these matters

- Jenkins reads pipeline directly from SCM.
- Follows DevOps best practice (pipeline as code)



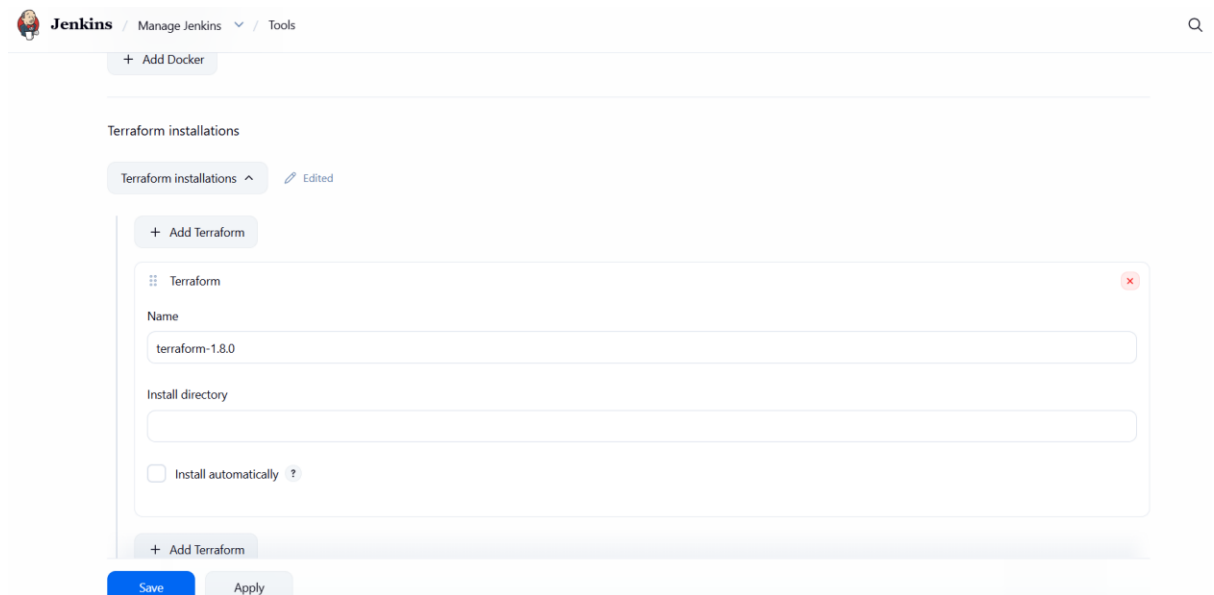
Install the Terraform Plugin

- Navigate to **Manage Jenkins > Plugins > Available plugins**.
- Search for **Terraform**, select it, and click **Install**.

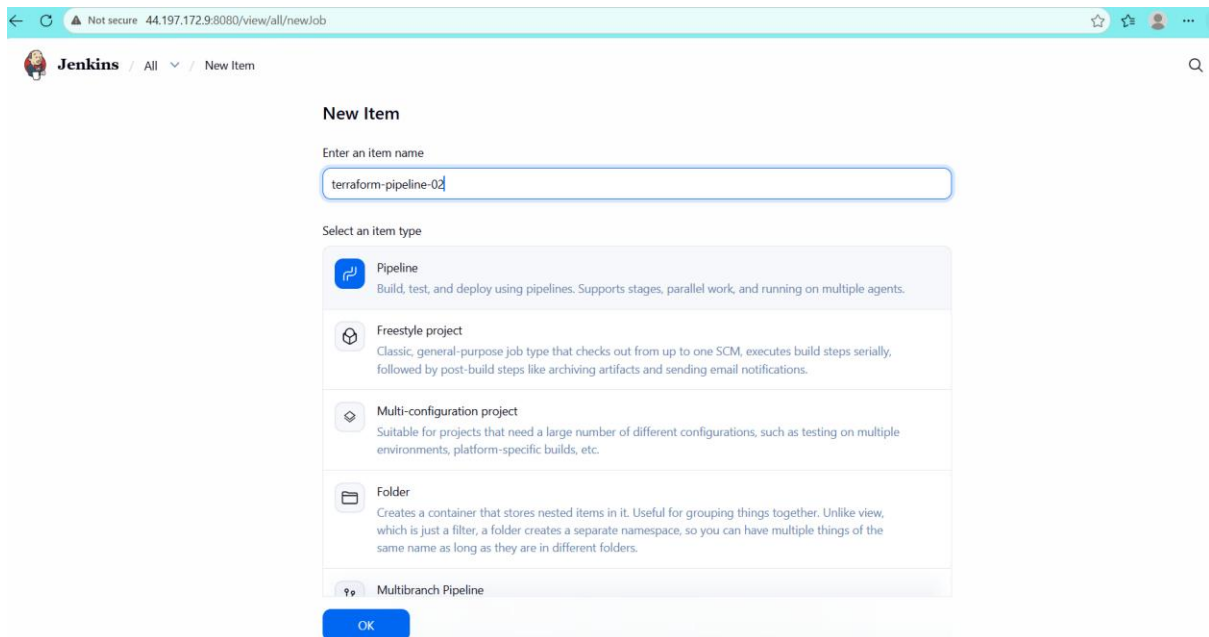


Configure the Terraform Tool in Jenkins

- Go to **Manage Jenkins > Tools (or Global Tool Configuration)**.
- Scroll down to the **Terraform** section and click **Add Terraform**.
- Give it a distinct name (e.g., terraform-1.8.0).
- Either check **Install automatically** to let Jenkins fetch the binary, or specify the path where the Terraform binary exists on the Jenkins server.



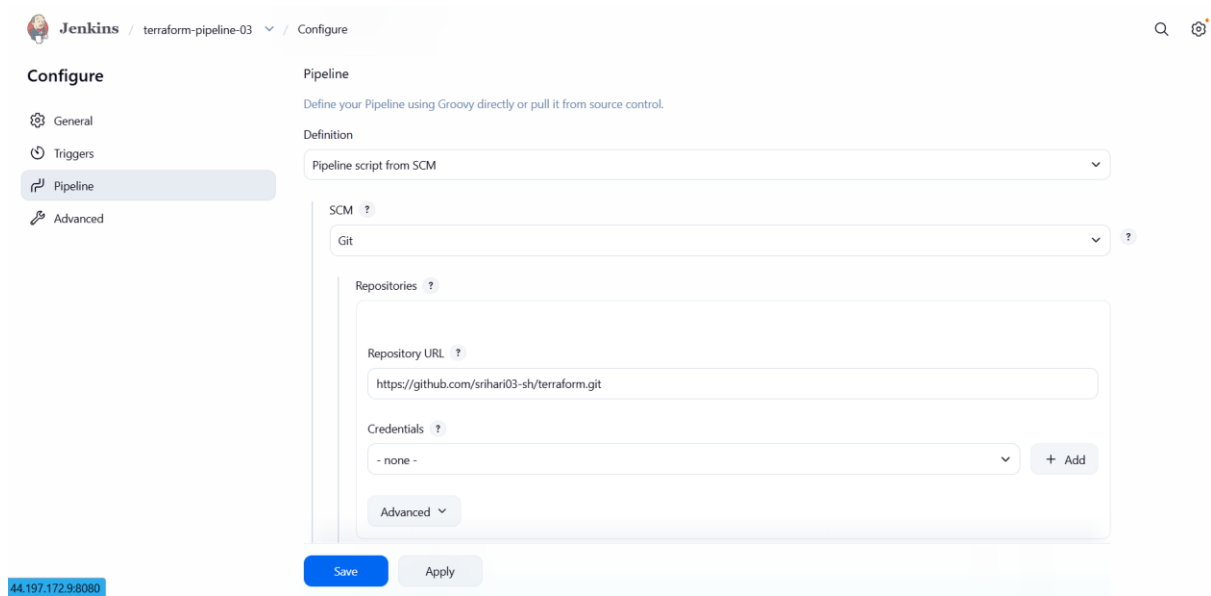
- Select new item
- Enter item name: terraform-03
- Select pipeline



- Definition: **Pipeline script from SCM**
- SCM: **Git**
- Repository URL:
- <https://github.com/srihari03-sh/terraform.git>

Why this matters

- Jenkins automatically fetches pipeline code from GitHub.

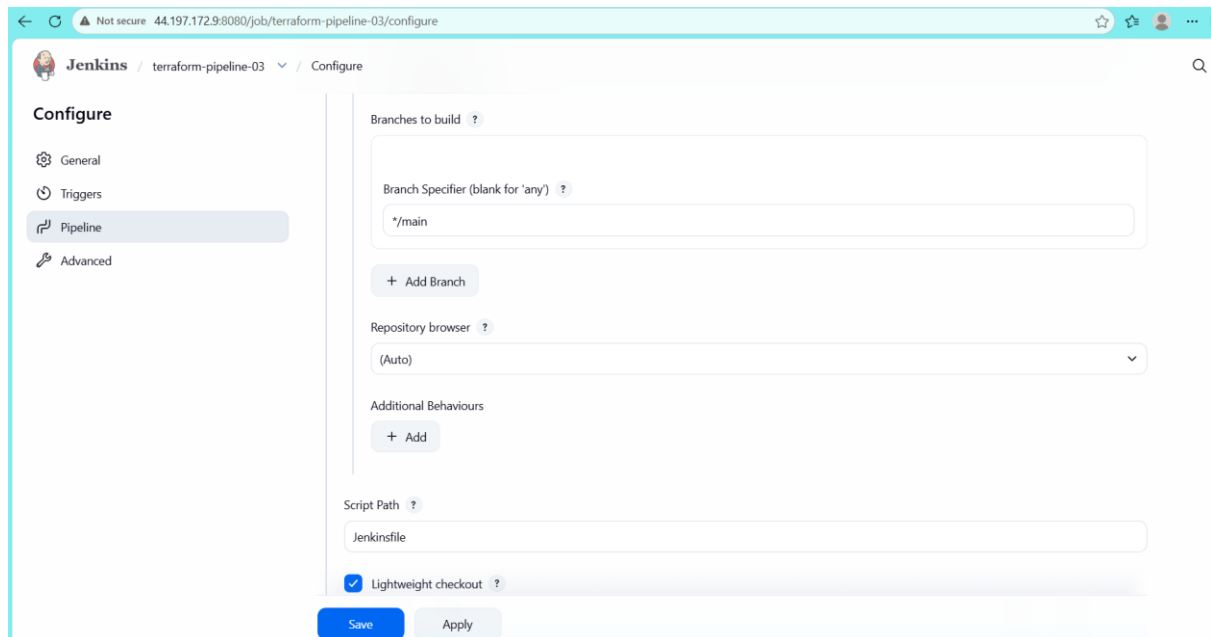


The screenshot shows the Jenkins configuration page for a pipeline named 'terraform-pipeline-03'. The left sidebar has tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced', with 'Pipeline' selected. The main area is titled 'Pipeline' and contains the following fields:

- Definition:** A dropdown menu set to 'Pipeline script from SCM'.
- SCM:** A dropdown menu set to 'Git'.
- Repositories:** A section containing:
 - Repository URL:** A text field with the value 'https://github.com/srihari03-sh/terraform.git'.
 - Credentials:** A dropdown menu set to '- none -' with a '+ Add' button.
 - Advanced:** A dropdown menu.

At the bottom of the configuration area are 'Save' and 'Apply' buttons. A small blue box in the bottom left corner of the screenshot contains the IP address '44.197.172.9:8080'.

- Branch Specifier:
- */main
- Script Path:
- Jenkinsfile



This screenshot shows the 'Branches to build' section of the Jenkins configuration page. The browser address bar shows '44.197.172.9:8080/job/terraform-pipeline-03/configure'. The left sidebar is the same as the previous screenshot. The 'Branches to build' section contains:

- Branch Specifier (blank for 'any'):** A text field with the value '*/main'.
- + Add Branch:** A button to add a new branch specifier.
- Repository browser:** A dropdown menu set to '(Auto)'.
- Additional Behaviours:** A section with a '+ Add' button.
- Script Path:** A text field with the value 'Jenkinsfile'.
- Lightweight checkout:** A checkbox that is checked.

'Save' and 'Apply' buttons are at the bottom of the configuration area.

- Build is triggered manually.
- Jenkins performs:

1. Git checkout
2. terraform init
3. terraform plan
4. Manual approval
5. terraform apply

Result

- Resources created successfully.
- File written to /tmp/pets.txt.
- Random pet name generated.

Jenkins / terraform-pipeline-03

Status: ✔ terraform-pipeline-03

Stage View

Average stage times: (full run time: ~17s)

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout from SCM	Terraform Init	Terraform Plan	Terraform Apply	Declarative: Post Actions
#1 Sep 03 20:54 No Changes	363ms	124ms	442ms	940ms	686ms	466ms (paused for 13s)	68ms

Permalinks

- [Last build \(#1\), 22 sec ago](#)
- [Last stable build \(#1\), 22 sec ago](#)
- [Last successful build \(#1\), 22 sec ago](#)
- [Last completed build \(#1\), 22 sec ago](#)

Builds

Filter: /

Today

✔ #1 3:24 PM